When Less Is More: Trade-off Between Few-shot and Fully Supervised Named Entity Recognition

Cornelius Nielsen
cotn@itu.dk

Pietro Rebecchi
pier@itu.dk

Szonja Uley
szul@itu.dk

Thomas Broholt Knudsen
thkn@itu.dk

Abstract

In Named Entity Recognition (NER), where the aim is to recognize and label occurrences of entities in a body of text, few-shot learning is often used when labeled data is scarce or costly to obtain. Although supervised BERT nearly always performs the best, it is unclear how much data is needed before a standard supervised model consistently outperforms few-shot alternatives. In this study, we systematically compare ProtoBERT, a leading few-shot method, against a supervised BERT-Tagger across 18 settings varying in class diversity and support size using the Few-NERD dataset. We find that supervised models generally surpass the few-shot models around 2,000 training sentences, though this threshold varies significantly by entity complexity and distribution with regular supervised never outperforming in some scenarios. These findings provide guidance for choosing between few-shot and supervised NER models, depending on data availability, entity granularity, and domain requirements. Code available at https://github.com/CorneliusMN/Few_Nerds.git.

1 Introduction

Gathering and labeling data can be time consuming and costly. Therefore, few-shot learning models are often used when training data is limited or when the set of named entity labels evolves so frequently that retraining a fully supervised model becomes impractical.

The Few-NERD paper (**?**), which laid the groundwork for evaluating NER models in few-shot scenarios, reports that ProtoBERT - under its best setup - achieves an F1 score of 56.29 ± 0.79 on the Few-NERD dataset, making it a strong contender when labeled data is scarce. However, with sufficient training data a regular supervised BERT model achieves an F1 score of 68.88.

Yet it remains unclear when exactly a few-shot method, specifically ProtoBERT, becomes suboptimal compared to a standard supervised learning model. Our main contribution lies in identifying this boundary, which will help researchers make informed decisions about whether to spend more resources on gathering data or rely solely on a few-shot ProtoBERT approach.

To establish the volume of training data, measured in number of sentences, required for supervised learning to outperform few-shot learning, we replicate the few-shot model configuration achieving the highest F1 score in the Few-NERD paper. Specifically, among the few-shot learning methods presented in the paper, we select ProtoBERT in the INTER setting, which uses fine-grained labels from the Few-NERD dataset.

To ensure a fair comparison between the selected supervised and few-shot approaches, we train both models on comparable data and evaluate them at the same predefined intervals using identical evaluation sentences. While the supervised BERT-Tagger is evaluated in a within-domain setting, requiring label overlap between training and test sets, the few-shot ProtoBERT model operates in an out-of-domain (INTER) setting, making its performance particularly notable. We adopt this setup to remain consistent with the original paper.

Our findings show that the trade-off is problem-specific; however, the standard supervised BERT-Tagger outperforms the few-shot alternative ProtoBERT in most settings at approximately 2,000 sentences. The takeover is highly dependent on the classes' composition and the number of examples per class shown during each episode.

2 Experimental Setup

2.1 Data Setup

We use the Few-NERD dataset introduced in the original paper, as it is the largest human-annotated NER dataset containing a wide range of entity types

(8 coarse-grained and 66 fine-grained categories). Since our project can be seen as an extension or a more in-depth exploration of a specific aspect of the original paper, we want to use the same dataset to ensure a comparable and coherent experimental setup.

To evaluate the performance of both the within-domain BERT-Tagger model and the out-of-domain ProtoBERT model, we create new splits using the fine-grained labels from the Few-NERD dataset. For computational efficiency, we choose the coarse-grained categories *Person, Product* and *Organization*, since these make up a large part of the dataset (~60%) and also offer a variety of different fine-grained entity types.

For each of the selected categories, we create "single-splits" - each containing only the fine-grained labels belonging to its respective coarse-grained group. Additionally, we also create "combined-splits", where episodes are sampled from all three categories together. We take a few-shot episodic approach (Algorithm 1), where each episode is defined by the parameters n and k:

- n = number of classes (set to 1, 3, or 5);
- k = number of support examples per class (set to 1, 5, or 10).

The episodes are fed to the few-shot ProtoBERT model, which is trained on the training data and evaluated on development data every ~80 sentences. In contrast, the supervised BERT-Tagger is trained with access to both training and development support sets and then evaluated on the development query data every ~80 sentences.

The splits are designed to enable a direct comparison between few-shot generalization across domains and fully supervised performance within a consistent domain.

2.2 Model

The original paper proposed three few-shot learning methods: ProtoBERT, NNShot and StructShot. As ProtoBERT performed the best on average with INTER settings, we choose this model to compare with the regular supervised BERT model.

2.2.1 BERT-Tagger

For our base supervised model, we choose a BERT uncased model identical to the one used in the Few-NERD paper as we know the expected performance for this model on the dataset. Furthermore, this choice also facilitates a fairer comparison with our few-shot model choice as it is also based on BERT.

Algorithm 1: Generate Few-shot NER Episodes

```
Read label list from labels_file;
Filter labels by desired prefixes;
Load dataset from data_file with filtered labels;
Initialize sampler with parameters (N, K, Q = K);
for i ← 1 to num_episodes do
    (C, S, Q) ← sampler.next();
    class ← coarse-grained labels C;
    support ← dataset entries at indices S;
    query ← dataset entries at indices Q;
    Build episode = {types: C, support, query};
    Write episode to output file;
end for
```

For the BERT-Tagger, each input sentence $x = x_1, ..., x_n$ is passed through an encoder to obtain a sequence of embeddings $h = h_1, ..., h_n$, where h_1 represents the embedding for token x_1. The embeddings are then passed through a linear classification layer that maps each h_i to a distribution over the class labels.

Training is performed using cross-entropy loss and evaluation is computed every ~80 sentences from the query portion of the dev split to ensure that both models are evaluated on the same data. However, unlike ProtoBERT, which is trained in an out-of-domain setting, the BERT model has access to within-domain label types.

2.2.2 ProtoBERT

ProtoBERT is a method based on prototypical networks (**?**) with a BERT encoder. It operates in two steps:

- Prototypes creation: for each entity type, it computes a prototype that corresponds to the average of the embeddings of all support tokens for that type;
- Classification: for each query token, it calculates the L2 distance (Euclidean distance) between the query's embeddings and each prototype, assigning the label of the nearest prototype.

During training, the parameters are updated at each meta-task; during evaluation, the predictions are based on the closest prototype.

2.3 Machine Setup

To compare the time efficiency of the models, we use the same machine with a 48-core AMD EPYC 7443 CPU, 250 GiB of RAM, and 2× NVIDIA RTX 6000 Ada GPUs with 48 GiB each and time every run.

3 Results

3.1 Trade-off Point

After completing all 18 runs, we plot the total span F1 scores for the supervised and few-shot runs. We select three representative plots that illustrate the general trend observed in the other runs, which are available in the *Appendix*. The exception being the "single-type" run on the *actor* category where the few-shot model matched or outperformed the supervised counterpart throughout the entire run. Figure 1 shows the *product_1_10* run. The supervised model matches the performance at around 2,000 sentences.

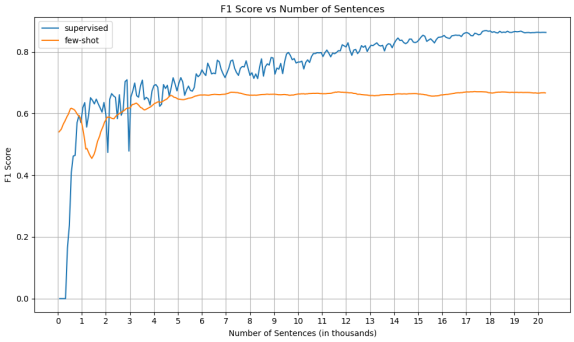

Figure 1: Span F1 for product_1_10 run

Figure 2 shows the *combined_1_5* run. The supervised model seems to match the performance around 10,000 sentences.

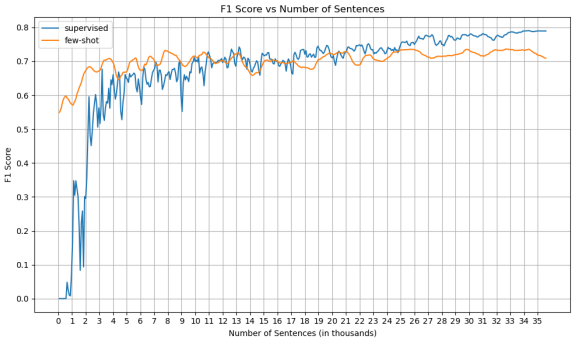

Figure 2: Span F1 for combined_1_5 run

Figure 3 shows the *combined_3_1* run. The models cross at about 2,000 sentences.

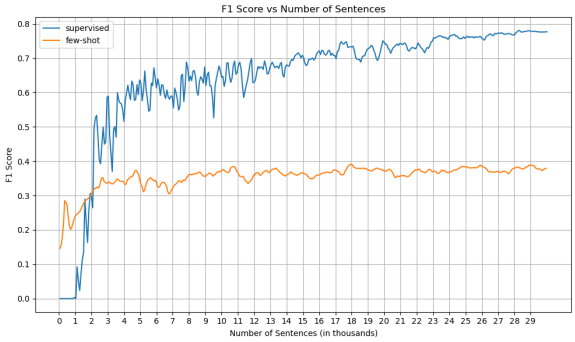

Figure 3: Span F1 for combined_3_1 run

Our findings compare supervised and few-shot learning methods for Named Entity Recognition (NER) across various N-way K-shot configurations and coarse-grained entity types. Notably, the few-shot models are evaluated on out-of-domain data, while the supervised models are evaluated on within-domain data, making the few-shot task inherently more challenging. In Table 1, the "Stable" column indicates the sentence count at which a model's performance plateaus, the "F1" column reports the F1 score at that stable point, and the "Takeover" column shows the sentence count at which the supervised model's performance surpasses that of the few-shot model.

Setup	Few-shot		Supervised		Takeover
	Stable	F1	Stable	F1	
organization_1_1	1,440	0.56	3,360	0.65	950
organization_1_5	5,510	0.65	9,440	0.74	600
organization_1_10	4,610	0.73	1,984	0.84	10,500
person_1_1	1,120	0.85	2,560	0.76	NA
person_1_5	1,660	0.93	7,600	0.90	NA
person_1_10	3,760	0.92	14,560	0.91	18,000
product_1_1	2,560	0.51	2,560	0.66	700
product_1_5	4,900	0.61	8,400	0.76	900
product_1_10	4,650	0.66	15,360	0.83	1,000
combined_1_1	5,920	0.66	6,720	0.67	6,500
combined_1_5	14,860	0.75	25,040	0.75	23,000
combined_1_10	4,570	0.72	6,560	0.70	14,500
combined_3_1	4,630	0.38	22,960	0.76	1,500
combined_3_5	2,360	0.46	5,600	0.65	2,200
combined_3_10	5,870	0.50	8,160	0.69	2,000
combined_5_1	12,800	0.30	18,880	0.74	1,000
combined_5_5	1,530	0.40	4,080	0.67	2,200
combined_5_10	260	0.39	5,280	0.61	1,400

Table 1: Comparison of performance between Few-shot and Supervised NER models across different setups

3.2 Time Efficiency

As shown in Table 2, for all runs the few-shot model takes longer, which can be expected as the ProtoBERT model is a BERT-based model with an additional layer to it.

On the other hand, it is worth considering that the few-shot model stabilizes at a much earlier stage of the training process regardless of the setup, so stopping the training at a set sentence count would

have smaller impact on the performance compared to that of the supervised method.

Setup	Few-shot			Supervised		
	real	user	sys	real	user	sys
product_1_10	113	164	18	55	36	7
combined_1_5	219	402	43	171	117	38
combined_3_1	137	238	31	135	90	29

Table 2: Comparison of real, user, and sys time (in minutes) between Few-shot and Supervised setups

4 Analysis

4.1 Total F1 Scores

Inspecting Table 1, few-shot learning demonstrates strong early performance, particularly in low-N, high-K setups. For example, *person_1_5* reaches an F1 of 0.93 using just 1,660 sentences, while supervised requires 7,600 sentences to approach a similar F1. In *person_1_1*, few-shot achieves 0.85 F1 with only 1,120 sentences, and the supervised model never surpasses it. These results suggest that when working with relatively uniform data and fewer entity types, few-shot models can provide substantial savings in labeled data requirements.

However, as task complexity increases — especially with more entity classes or diverse labels — supervised learning becomes more advantageous. For instance, in *combined_5_1*, a few-shot starts at a low F1 of 0.30, while supervised overtakes it at 1,000 sentences and eventually reaches 0.74. Similar patterns are seen in *combined_3_1* and *product_1_5*, where few-shot initially performs reasonably well but is surpassed early by supervised models with modest increases in data.

The point at which supervised learning overtakes few-shot learning varies across setups and can guide resource allocation decisions. In simpler scenarios such as *organization_1_1*, the takeover happens early at just 950 sentences. In more challenging or balanced conditions like *combined_1_5*, it occurs much later or it achieves a similar F1 score with more data. Notably, in some person entity setups, such as *person_1_1* and *person_1_5*, supervised models never surpass the few-shot performance, emphasizing that task characteristics significantly influence outcomes.

Looking at the plots, we see significant F1 increases in some places for the supervised model. An example being the *combined_3_1* run at around 1,500 sentences. Similarly we also notice some major dips in performance such as around 1,000 sentences in the *combined_1_5* run. Although not as big, the few-shot model does have some variance in its performance. To analyze this more thoroughly, we print the individual F1 scores for each fine-grained category.

4.2 Individual F1 Scores

For the few-shot model, we were unable to implement a span-based F1 measure that evaluates on each fine-grained category every ~80 sentences. However, as we have both the token-based and the span-based F1 scores for the fine-grained labels with the supervised model, we can see if the trend for these metrics align (*Appendix* section 6.3.3). Overall, the plots are very similar, with the main difference being the magnitude of variation. As the span-based metric is more strict, the magnitude of variation also gets larger. Nonetheless, for the purpose of analyzing which labels contribute to shifts in model performance, the standard token-based F1 computations for the few-shot model are sufficient. Moving forward, we use only the token-based F1 for consistency, since the token-based and span-based evaluations seem to follow the same trend.

We begin by examining the *product_1_10* run for the few-shot model, where a noticeable drop in performance occurs around 1,500 sentences. To investigate this, we plot the token-based F1 score (Figure 4).

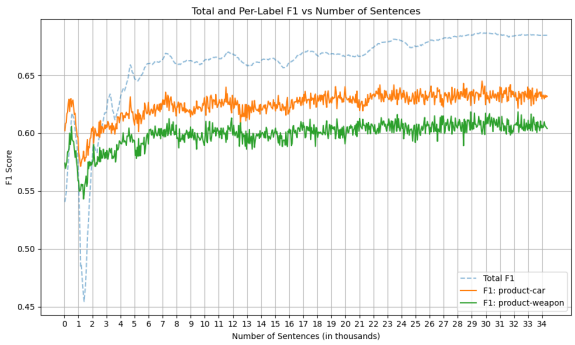

Figure 4: Token-based F1 for product_1_10 Few-shot run

From this figure, we can see that the performance trends of both fine-grained categories in the evaluation decline in parallel. The performance drop is therefore not due to a single category under-performing, but rather the model under-performing over all categories. One possible explanation lies in the setup of our training. The few-shot model is evaluated on labels it has not seen during training,

which helps with generalization rather than memorizing. Another factor is the characteristics of a prototype model: if the prototype for a specific label is inaccurate as a result of the support set shown at a given step in training, it can cause systematic misclassification across all categories, since examples may be falsely assigned to a incorrect prototype. As such, misclassifications are distributed more evenly, leading to uniform decreases and increases in performance. This behavior of small increases or decreases in performance across all groups is a general tendency seen throughout all runs available in the *Appendix*.

Next, we inspect the supervised runs first looking at the *combined_1_5* run as this contains both a dramatic increase and decrease in performance around 1,500 sentences. The F1 across all categories is seen in Figure 5.

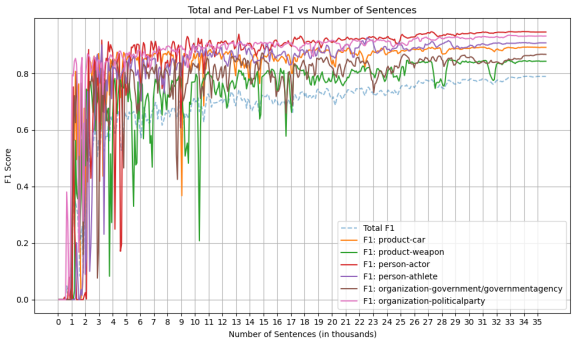

Figure 5: Token-based F1 for product_1_5 Supervised run

It can be seen that the total F1 fluctuation appears to be mainly due to large oscillations in the F1 for the *product* classes (*product-car* and *product-weapon*). This is expected behavior as the *product* category is the worst performing class in the Few-NERD study. The performance swings become less dramatic as the model is trained on more sentences, which can be explained by the within-domain setting for the supervised model: with increased exposure, the model learns more consistent patterns, resulting in reduced variability.

The larger variation in performance seen in the supervised model as opposed to the few-shot, can be explained by its sensitivity to the specific support-query distribution. Unlike the few-shot model which is designed to generalize across unseen labels, the supervised model may overfit to frequently occurring patterns in the training data. It is also worth noting that while the *product* categories are the worst performing, some other classes such as *person-actor* show consistently high performance across both models. This suggests that some entities are easier for the models to learn, likely due to clearer contextual cues or a more distinct sentence pattern.

5 Limitations and Conclusion

Despite the valuable findings, our study has a few limitations. Although we intentionally preserve the within-domain vs. out-of-domain distinction to remain consistent with the Few-NERD setup, this introduces a mismatch between the training data that may affect comparability. Furthermore, our evaluation is limited to a subset of coarse-grained categories, which may not generalize to all real-world NER scenarios/entity combinations. This is due to computational limitations, as it was not possible to test our hypothesis on all existing labels in the Few-NERD paper. Lastly, while we saw some alignment between span- and token-based F1 for supervised models, few-shot evaluation relies solely on token-level F1 due to implementation constraints, potentially affecting the preciseness of the evaluation.

Overall, few-shot models excel when data is scarce, the task involves few entity types, and generalization across domains is important. They provide faster performance gains in low-resource settings. Supervised models, on the other hand, benefit from more data and scale better in complex, multi-class environments.

When deciding which method to apply, it is important to consider the availability of labeled data, the domain specificity of the task, and the number and diversity of entity types. The results discussed in the *Analysis* section suggest that there is no one-size-fits-all answer: the optimal choice depends on balancing data annotation costs, task complexity, and the expected domain variability. This is highlighted by our finding with the crossover point occurring as early as 600 sentences in simple cases or requiring over 23,000 with more complex setups and entities. Moreover, the faster convergence of few-shot models makes them attractive for prototyping in smaller data environments, while the stronger long-term performance of supervised models justifies their use in large data environments.

6 Appendix

6.1 Group contributions

Our group met every Tuesday and Thursday to sit and work together on all elements of the project. All decisions were made together after discussions and we all share equal responsibility.

6.2 Chatbot usage

- Plot configuration
- Print statements aesthetics
- LaTeX table and figure layout suggestions

6.3 Additional results

6.3.1 Single-split Performance Comparison: BERT-Tagger vs ProtoBERT

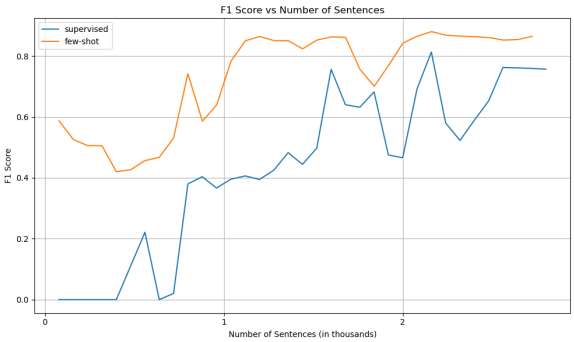

Figure 6: Person_1_1

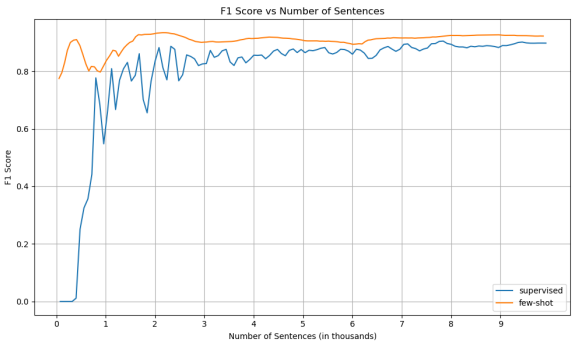

Figure 7: Person_1_5

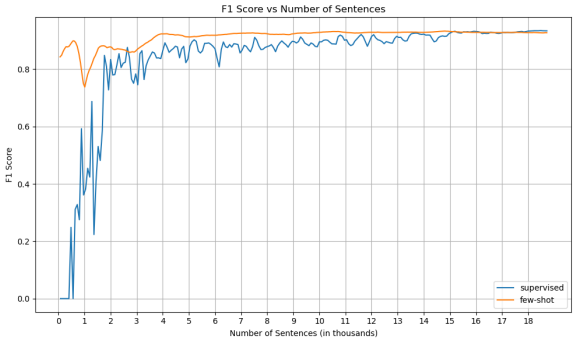

Figure 8: Person_1_10

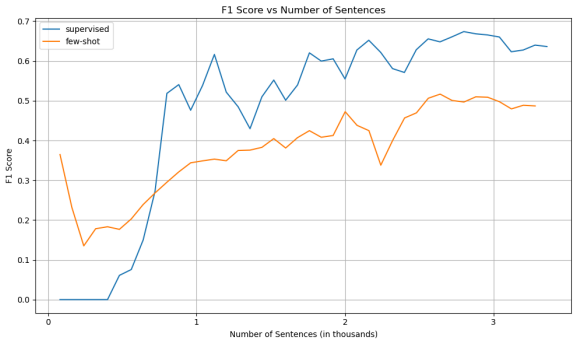

Figure 9: Product_1_1

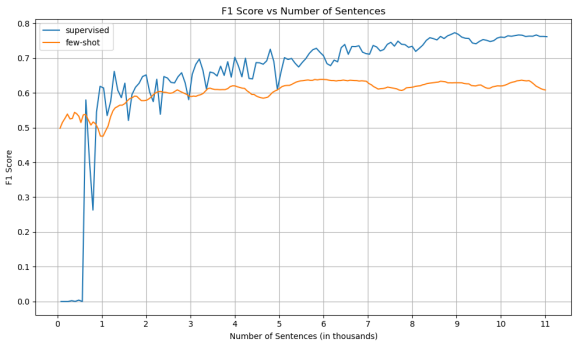

Figure 10: Product_1_5

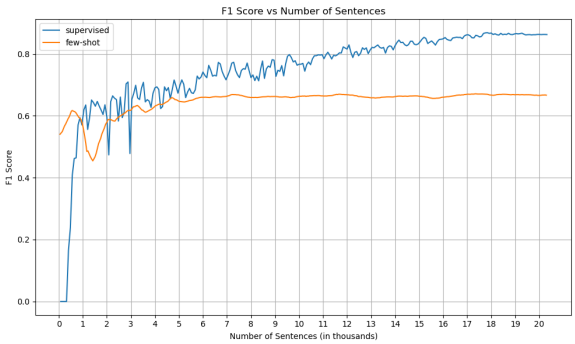

Figure 11: Product_1_10

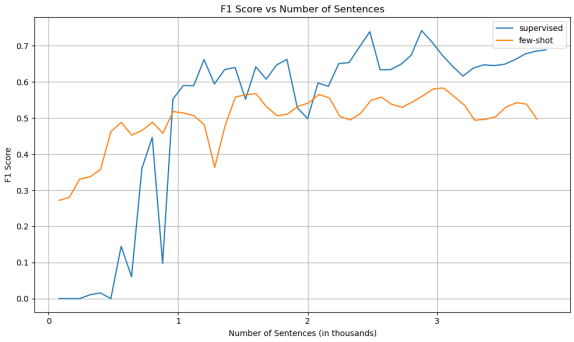

Figure 12: Organization_1_1

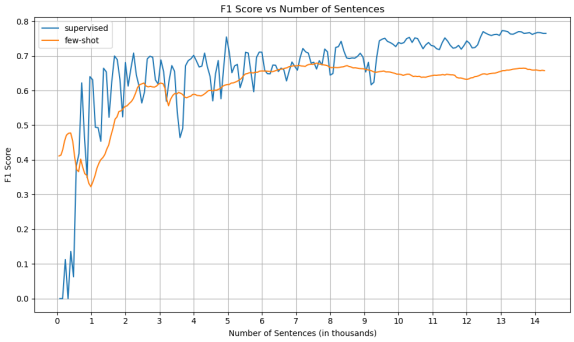

Figure 13: Organization_1_5

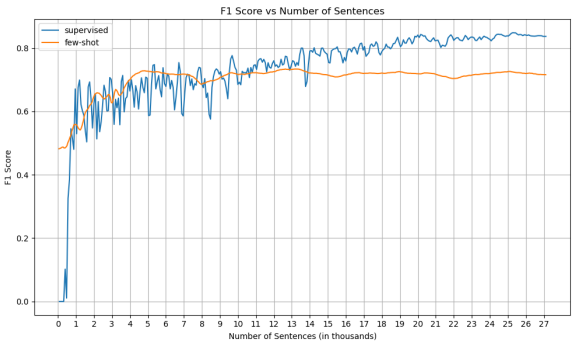

Figure 14: Organization_1_10

6.3.2 Combined-split Performance Comparison: BERT-Tagger vs ProtoBERT

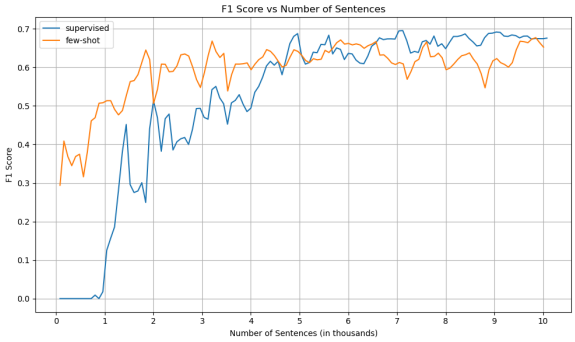

Figure 15: Combined_1_1

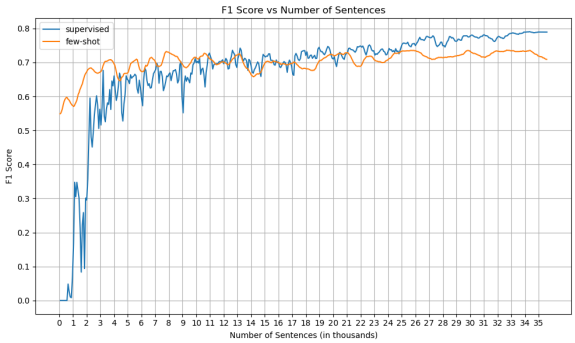

Figure 16: Combined_1_5

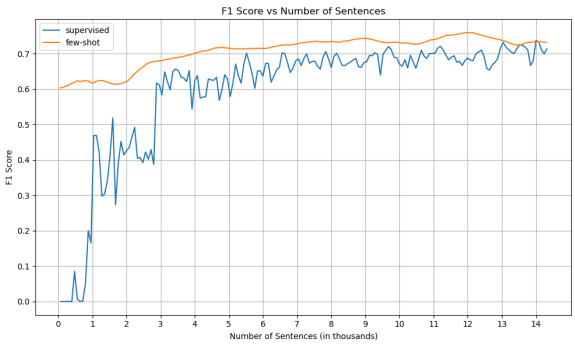

Figure 17: Combined_1_10

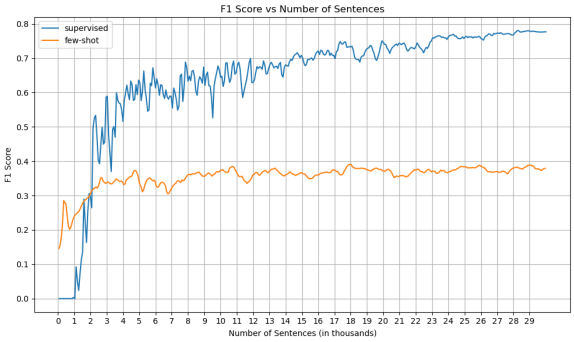

Figure 18: Combined_3_1

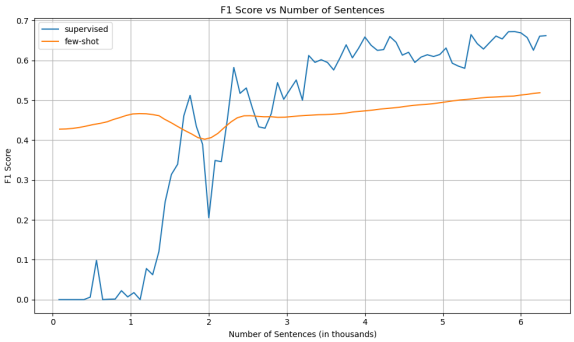

Figure 19: Combined_3_5

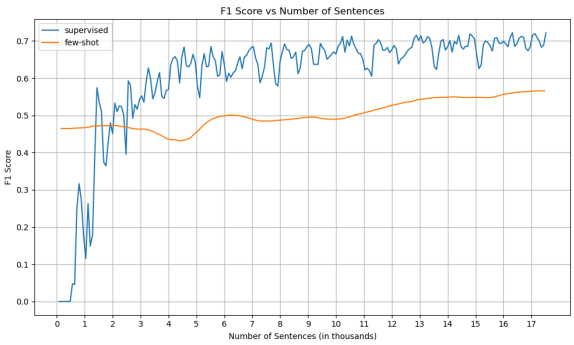

Figure 20: Combined_3_10

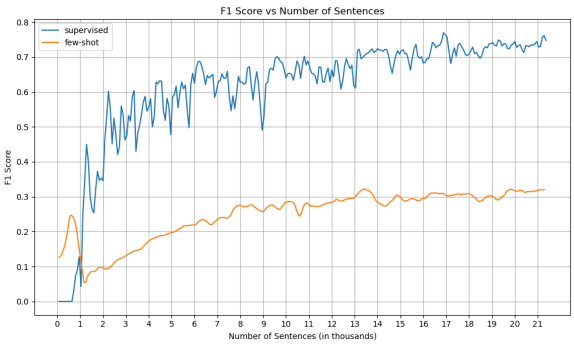

Figure 21: Combined_5_1

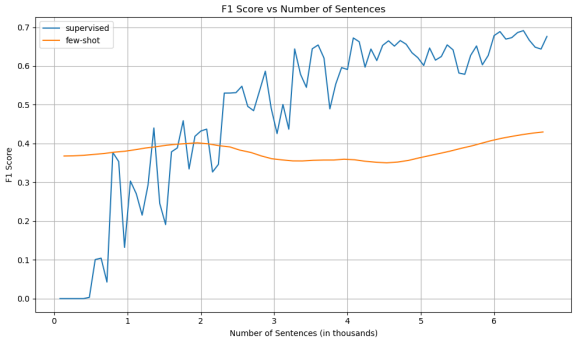

Figure 22: Combined_5_5

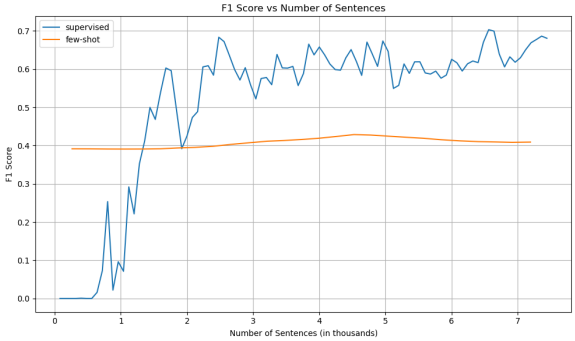

Figure 23: Combined_5_10

6.3.3 Comparison of Individual Span F1 vs Token-based F1 Analysis for Fine-grained Labels

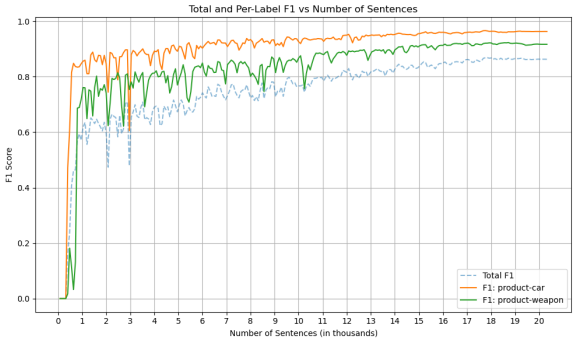

Figure 24: Product_1_10 with Individual Token-based F1

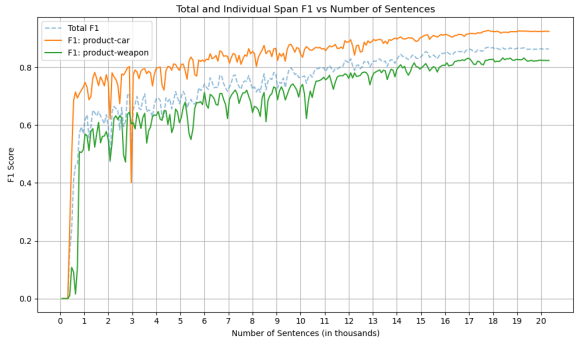

Figure 25: Product_1_10 with Individual Span F1

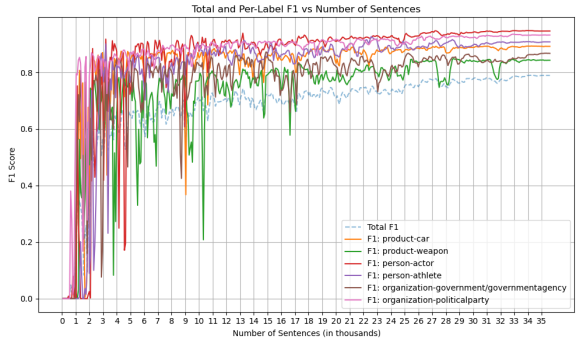

Figure 28: Combined_1_5 with Individual F1 BERT-Tagger

6.3.4 Impact of Individual Token-based F1 on the Total Span F1 Score

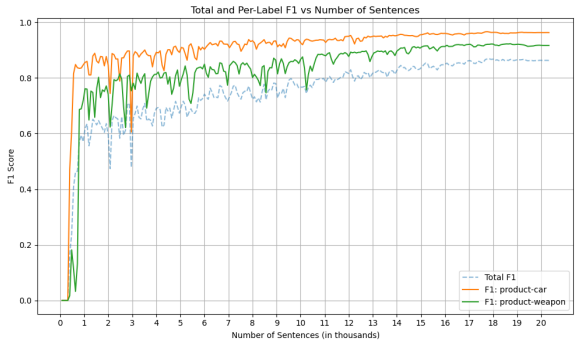

Figure 26: Product_1_10 with Individual F1 BERT-Tagger

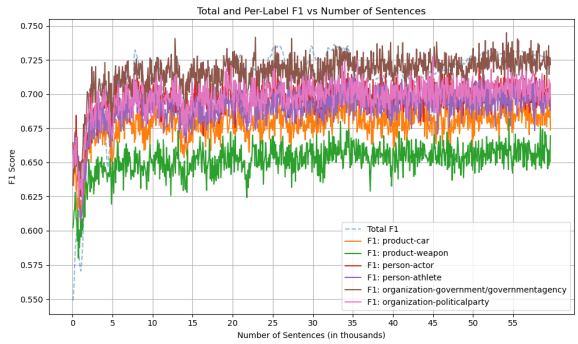

Figure 29: Combined_1_5 with Individual F1 Proto-BERT

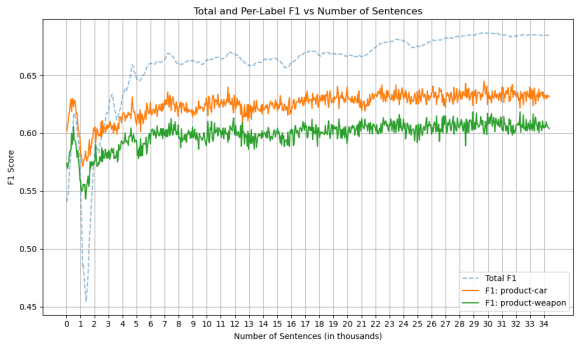

Figure 27: Product_1_10 with Individual F1 ProtoBERT

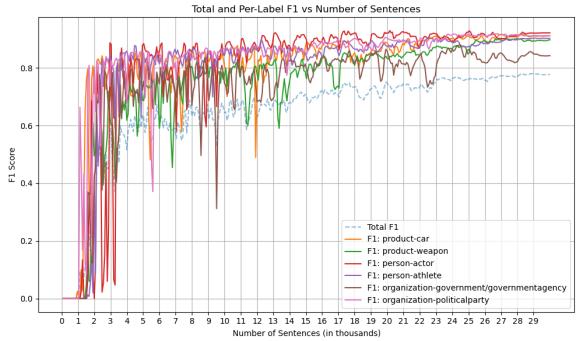

Figure 30: Combined_3_1 with Individual F1 BERT-Tagger

Figure 31: Combined_3_1 with Individual F1 Proto-BERT